

IplaceX

enovus

Alexis Rosales González
PROGRAMACION 2
BIMESTRE 5

Aplicación de Gestión de Medidores Domiciliarios.

Introducción

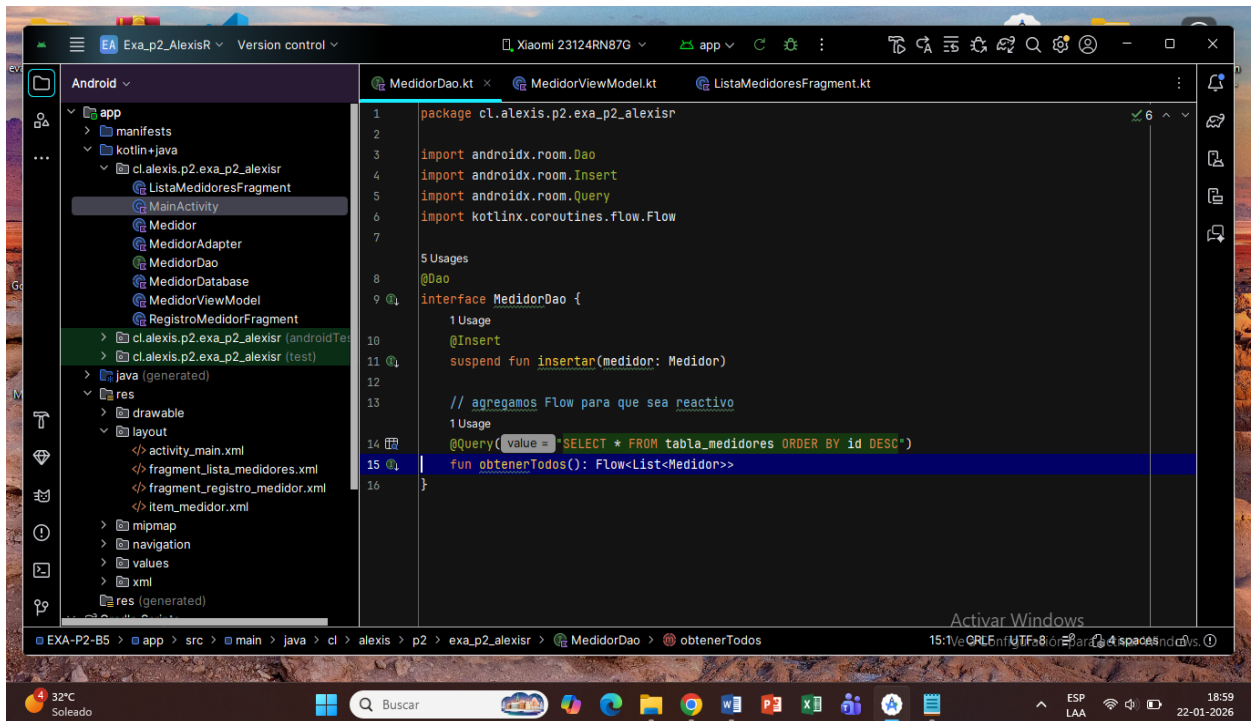
El presente proyecto consiste en el desarrollo de una aplicación móvil nativa en **Kotlin** para la gestión y registros de consumos de medidores domiciliarios (Luz, Agua y Gas). El objetivo fue implementar una solución técnica que cumpla con los requisitos funcionales de captura de datos, y también siga los estándares modernos de arquitectura en Android, como la persistencia de datos local, la gestión eficiente del ciclo de vida de los componentes y la reactividad de la interfaz. A continuación, se detallan los hitos técnicos más relevantes que permitieron alcanzar una aplicación escalable y además con una experiencia de usuario fluida en dispositivos físicos.

1. Implementación de persistencia reactiva con Room y Flow.

Para asegurar que los datos no se perdieran al cerrar la app, implemente una base de datos SQLite usando la biblioteca Room. Lo más importante fue la decisión de usar Flow en el DAO. A diferencia de una lista estática, lo que hace el Flow permite que la base de datos sea reactiva, es decir que en cuanto inserto un nuevo registro, Room notifica de forma automática a la interfaz sin que se tenga que refrescar la pantalla manualmente.

```
@Query("SELECT * FROM tabla_medidores ORDER BY id DESC")
```

```
fun obtenerTodos(): Flow<List<Medidor>> **// Esto garantiza que el usuario  
siempre vea la información más reciente al instante.//**
```



“Figura 1: interfaz MedidorDao implementando Flow para permitir la actualización reactiva de la lista de medidores desde la base de datos SQLite.”

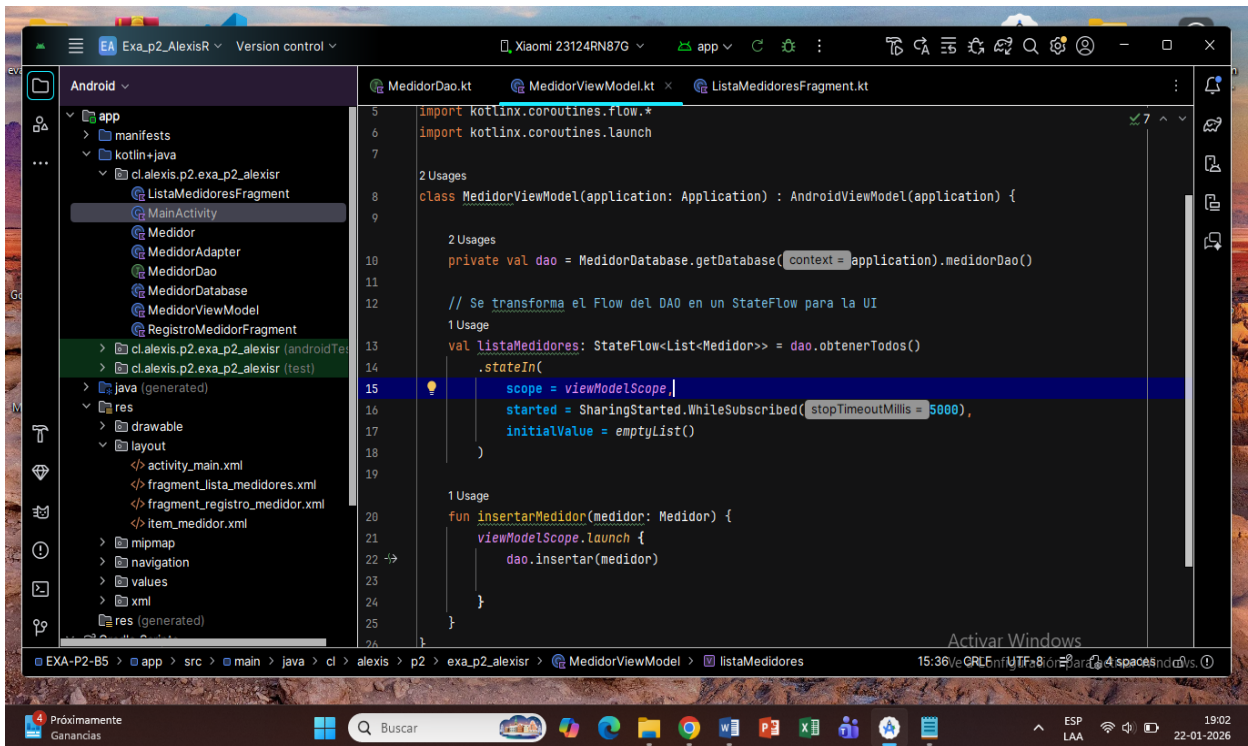
2. Arquitectura de Datos en el ViewModel.

Se utilizó el patrón de diseño ViewModel para separar la lógica de negocio de la interfaz de usuario. Aquí se implementó un stateFlow que consume el flujo del DAO. Lo configuré con SharingStarted.WhileSubscribed(5000), lo que permite que la app ahorre recursos al dejar de procesar datos si la pantalla no está visible, pero manteniendo el ultimo estado listo para cuando el usuario regrese a la app.

```
val listaMedidores: StateFlow<List<Medidor>> = dao.obtenerTodos()
```

```
.stateIn(scope = viewModelScope, ...)
```

****// en esta sección se utilizó el operador StateIn para transformar el Flow que viene de la base de datos Room en un StateFlow que la interfaz de usuario puede consumir de manera segura.//****



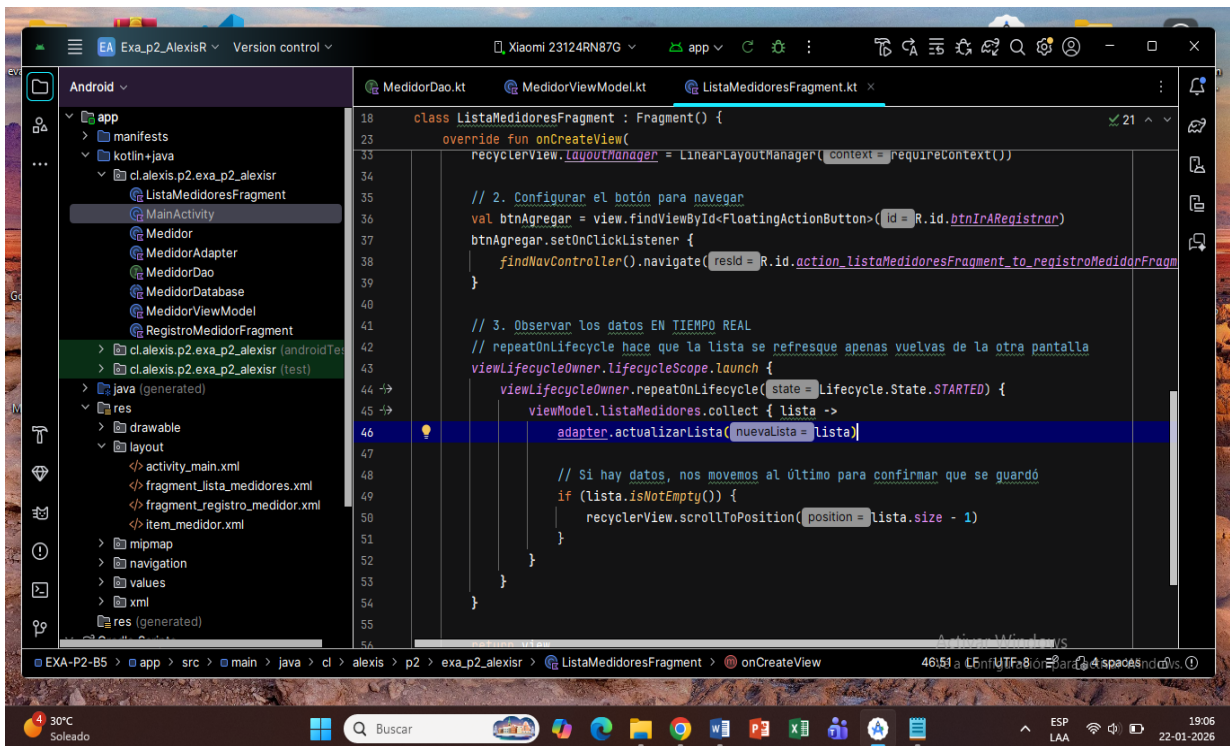
“Figura 2: Implementacion de MedidorViewModel donde se transforma el flujo de datos en un StateFlow, gestionando el estado de la UI de forma segura según el ciclo de vida.”

3. Sincronización de la interfaz con repeatOnLifecycle.

Un desafío crítico fue que la lista se actualizara apenas volvía de la pantalla de registro. Para solucionarlo, utilicé la API `repeatOnLifecycle`. Con esto me aseguré que el fragmento solo recolecte datos cuando la aplicación esta en primer plano, así evitando fugas de memoria y errores si la app intenta actualizarse estando en segundo plano.

```
viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {  
    viewModel.listaMedidores.collect { lista -> adapter.actualizarLista(lista) }  
}
```

****//** Esto se encarga de actualizar la interfaz en tiempo real al observar los cambios en la base de datos mediante el **repeatOnLifecycle**. Se utiliza este fragmento para recolectar los datos (collect) de forma segura.**//****

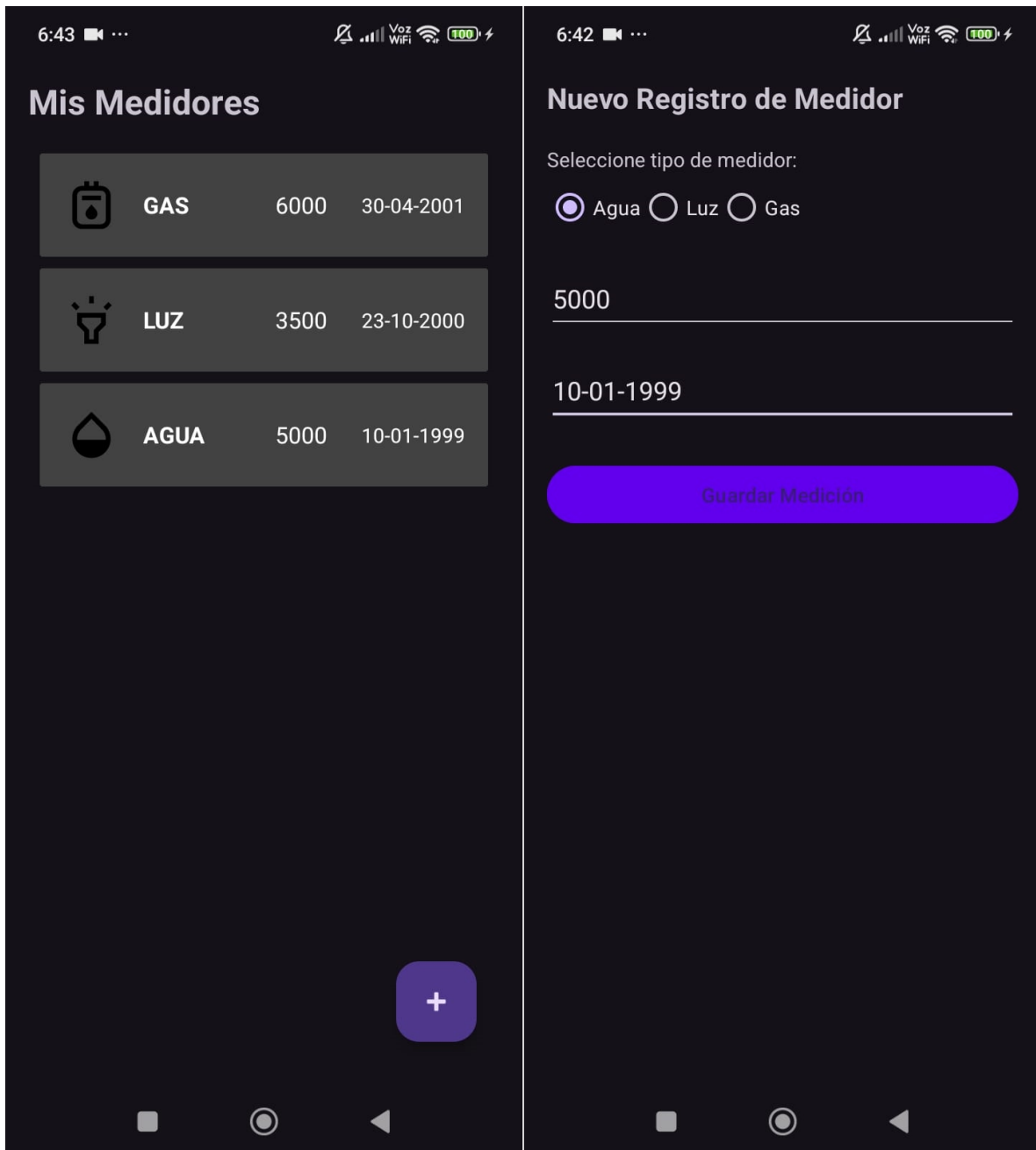


“Figura 3: Fragmento de código en el fragment donde se observa la lista de medidores utilizando repeatOnLifecycle para garantizar la actualización en tiempo real.”

4. Visualización eficiente con RecyclerView y Adaptadores.

Para mostrar los registros de forma fluida, se configura un RecyclerView. Se crea un adaptador personalizado que vincula de forma dinámica cada objeto Medidor con su respectivo diseño visual. Un detalle importante fue la lógica para asignar iconos específicos según el tipo de medidor, cumpliendo con el requisito de usar al menos 3 recursos drawables.

```
fun actualizarLista(nuevaLista: List<Medidor>) {  
    this.listaMedidores = nuevaLista  
    notifyDataSetChanged()  
} **// este código dentro del adaptador es el que refresca visualmente la lista cada  
vez que se detecta un cambio en la base de datos. al ejecutarse la función  
notifyDataSetChanged(), el RecyclerView redibuja los elementos en la pantalla  
permitiendo que el nuevo registro aparezca de forma fluida.**//
```



“Figura 4: Interfaz final de la aplicación ejecutándose en dispositivo físico xiaomi, mostrando el RecyclerView con los iconos correspondientes a Agua, Luz y Gas”.

5. Arquitectura de Navegación y Gestión de Fragmentos.

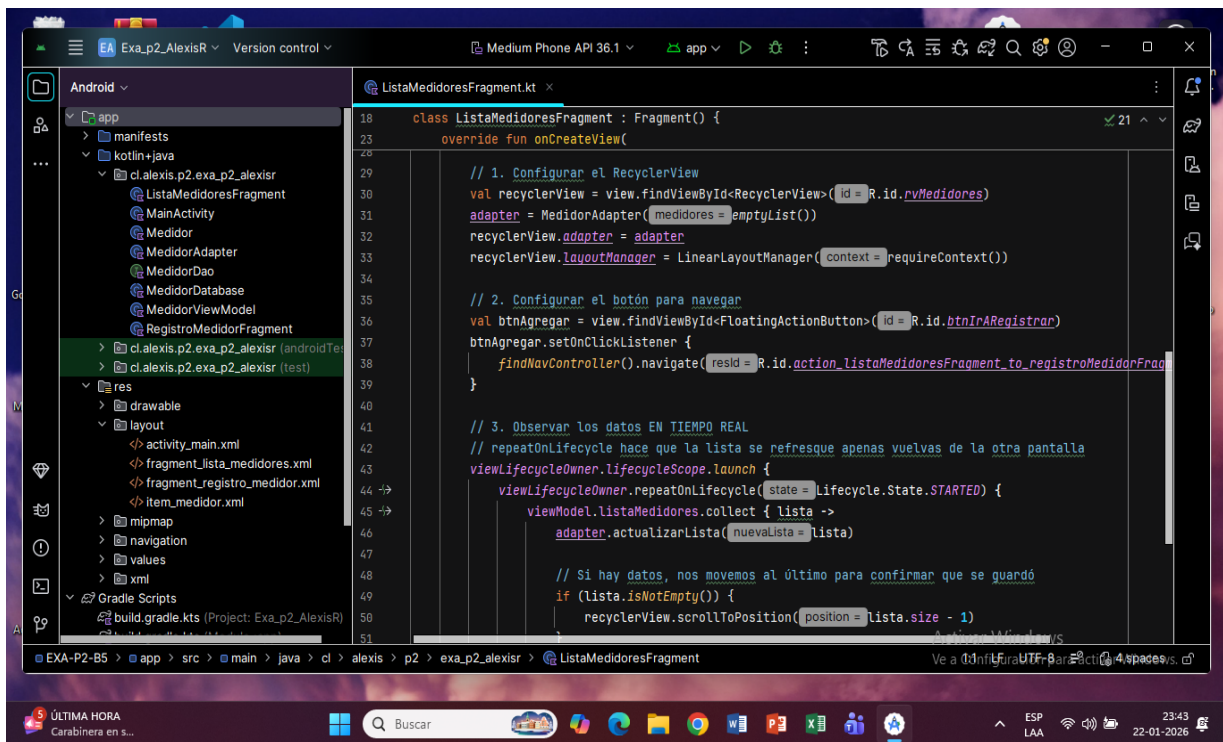
Para gestionar el flujo entre pantallas se implementa el componente Navigation de jetpack. Esta arquitectura permite que la transición desde la lista principal hacia el formulario de registro sea de forma fluida y estandarizada. Al usar un controlador de navegación, la aplicación maneja automáticamente las pantallas, permitiendo que el usuario regrese a la lista de medidores con el botón “atrás” sin tener errores de cierre.

```
val action =
```

```
ListaMedidoresFragmentDirections.actionListaMedidoresFragmentToRegistroMedidorFragment()
```

```
findNavController().navigate(action)
```

****// En esta sección se utiliza un componente NavController para realizar la navegación hacia el fragment de registro, asegurando una transición segura entre pantallas/****



“Figura 5: Implementación del controlador de navegación en el Fragment principal para gestionar el flujo hacia el formulario de registro de manera segura.”